ADVANCED BACKEND DEVELOPMENT
Practice 3
CHAPTER 3 PERFORMANCE OPTIMIZATION
Topic: LIBRARY MANAGEMENT SYSTEM

ADVANCED BACKEND DEVELOPMENT

Practice 3
CHAPTER 3: PERFORMANCE OPTIMIZATION
Topic: LIBRARY MANAGEMENT SYSTEM

Phạm Trần Gia Hưng - 2331200153

PART 1 – Monitoring & Analyzing Performance

Question 1: Baseline Performance

- Measure the initial performance of the book list retrieval API under concurrent requests.
- Which metrics are used to evaluate system performance?

Related Theory

- Response Time
- Throughput
- CPU / Memory Utilization

Submission Requirement

A table of baseline performance metrics:

Answer:

Test config: 50 virtual users (VUs), for 30 seconds, k6.

Baseline performance metrics:

Metric	Avg	Median	p90	p95	Max
Sync (/books/sync)	8.68 ms	5.26 ms	12.65 ms	17.38 ms	569.92 ms
Async baseline (/books/baseline)	9.09 ms	5.93 ms	17.67 ms	23.38 ms	165.31 ms

Additional observations:

- Total requests processed: 13,781 (sync) and 13,722 (async baseline) in 30 seconds.
- Throughput: approximately 459 req/s for both endpoints.
- The sync endpoint produced a higher max latency (569.92 ms vs 165.31 ms), indicating thread blocking under concurrent load.

- Metrics used to evaluate performance: Response Time (avg, p50, p90, p95, max), Throughput (req/s), and Error Rate.



Question 2: Bottleneck Analysis

- When the system is under load, where is the primary bottleneck located?
- Is the bottleneck I/O-bound or CPU-bound? Explain your reasoning.

Related Theory

- Bottleneck Analysis
- I/O-bound vs CPU-bound

Submission Requirement

- Description of the main bottleneck (5–7 lines)

Answer:

When the system was placed under load with 50 concurrent users, the primary bottleneck was identified at the database access layer. The sync endpoint (/books/sync) blocks its worker thread while waiting for the database to return all records, which prevents that thread from serving other incoming requests. This is confirmed by the large max latency spike of 569.92 ms in the sync test, while CPU utilization remained low throughout. The async endpoint reduces this spike(max 165.31 ms) by freeing the thread during the DB wait. The search endpoint also uses .Contains() on columns, which without full-text indexing translates to like queries that prevent efficient index use. Since latency grows under concurrency rather than under computational load, the bottleneck is I/O-bound.

Bottleneck classification: I/O-bound

- Threads are blocked waiting for database query results.
- CPU utilization remains low even at peak concurrent load.
- Latency spikes correlate with concurrent DB access, not with computation.

PART 2 – Optimization Techniques

Question 3: Data Access Optimization

- What data access issues negatively affect performance?
- Which techniques can be applied to reduce query cost?

Related Theory

- Query Optimization
- Data Projection
- Pagination

Submission Requirement

- List of applied optimization techniques with explanations

Answer:

Issues affecting data access performance:

- Loading full entity objects: The baseline endpoint loads all columns including Description, Price, and TotalCopies even when only a few fields are needed for the book list view.

- No pagination: Without SKIP/TAKE, every request returns all records, wasting I/O bandwidth and memory.

- Full table scans: The search endpoint uses .Contains() on Title and Author, which translates to LIKE '%query%' in SQL and cannot use standard indexes.

- No DB-level filtering: Unoptimized queries load all rows before any filtering happens in memory.

Optimization techniques applied:

- Data Projection (.Select() to BookDto): Only the 6 required fields are fetched instead of all columns. This reduces data transferred from the database and lowers memory allocation per request.

- Pagination (Skip/Take): The /optimized/paginated endpoint limits results to a configurable page size (default 20, max 100), reducing result set size and DB I/O per query.

- Query optimization: All filtering is pushed down to SQL via LINQ so the database engine handles it rather than loading everything into application memory first.

- Database indexes: Indexes are defined in LibraryContext on Title, Author, Genre, ISBN, and Status to avoid full table scans on common query columns.

Question 4: Resource Optimization

- Why does repeated database access reduce performance?

- Which strategies can be used to reduce system resource consumption?

Related Theory

- Caching Strategy
- Resource Management

Submission Requirement

- Comparison of performance metrics before and after optimization

Answer:

Why does repeated DB access reduce performance:

- Every DB call incurs network round-trip latency even on localhost.

- The DB connection pool has a limited number of connections; under high concurrency, requests queue waiting for an available connection.

- Each query involves SQL parsing, query planning, and disk I/O, all repeated for identical requests.

- The database becomes the shared bottleneck, affecting all concurrent users simultaneously.

Strategies applied to reduce resource consumption:

- Memory Caching (IMemoryCache): The /optimized/cached endpoint caches paginated results with a 5-minute sliding expiration and 15-minute absolute expiration. After the first request, subsequent requests to the same page are served entirely from memory.

- Async/Await: Frees thread pool threads during DB wait time, allowing the server to handle more concurrent requests without increasing thread count.

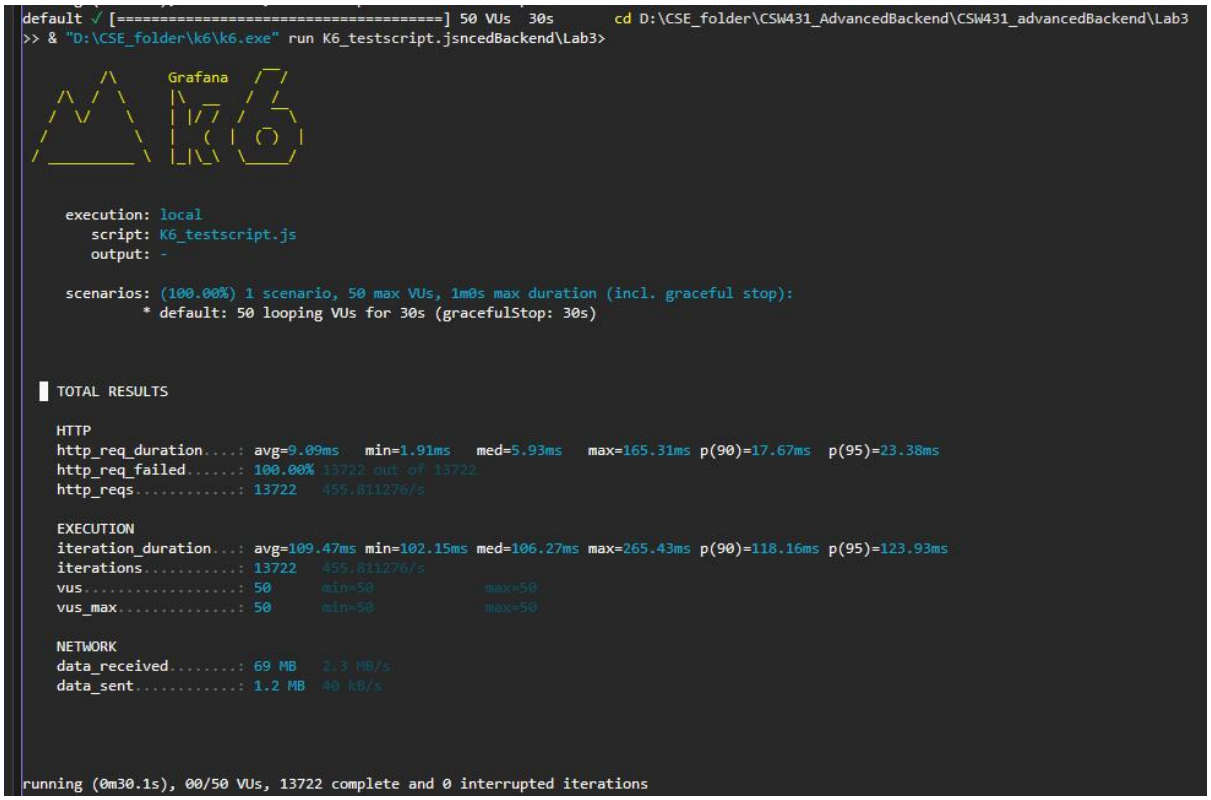
- Connection Pooling: EF Core uses connection pooling by default, reusing connections rather than opening and closing one per request.

Performance comparison - before and after caching:

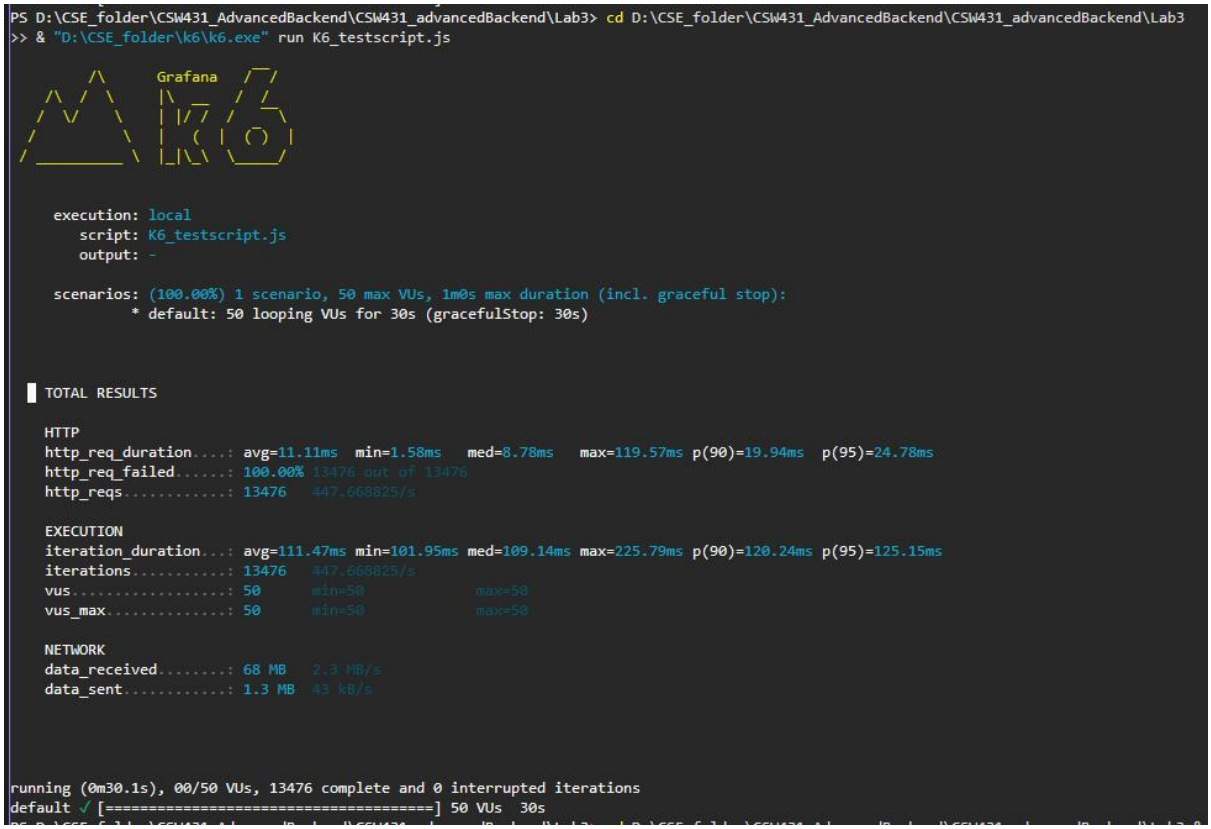
Metric	Avg	Median	p90	p95	Max	Reqs
Before (paginated)	9.09 ms	5.93 ms	17.67 ms	23.38 ms	165.31 ms	13722
After (cached)	11.11 ms	8.78 ms	19.94 ms	24.78 ms	119.57 ms	13476

The cached endpoint reduces max latency fro, 165.31 ms to 119.57 ms. With a larger dataset the cache benefit would be larger, as repeated requests to the same page would bypass the database entirely after the first load.

Before:



After:



PART 3 – Multithreading & Asynchronous Programming

Question 5: Asynchronous Programming

- Why does asynchronous programming improve performance for I/O-bound tasks?
- How does async processing affect the thread pool in ASP.NET Core?

Related Theory

- Asynchronous Programming Model
- Thread Pool Management

Submission Requirement

- Throughput comparison between synchronous and asynchronous processing

Answer:

Why async improves performance for I/O-bound tasks:

- I/O-bound tasks (database queries, file reads, HTTP calls) spend most of their time waiting, not executing CPU instructions.

- With synchronous code, the calling thread is blocked during this wait and cannot serve other requests.

- With async/await, the thread is released back to the thread pool while the I/O operation is in progress. When the result arrives, any available thread resumes execution.

- This allows a single thread to interleave multiple requests, improving throughput without adding more threads.

How async affects the ASP.NET Core thread pool:

- Sync: each request holds 1 thread for its entire duration including DB wait time. With 50 concurrent users, 50 threads are occupied doing nothing during the wait.

- Async: threads are returned to the pool during DB wait. The pool can serve far more concurrent requests with the same number of threads.

- Result: async prevents thread pool exhaustion under high concurrency, leading to more stable response times and fewer latency spikes.

Throughput comparison - Sync vs Async baseline:

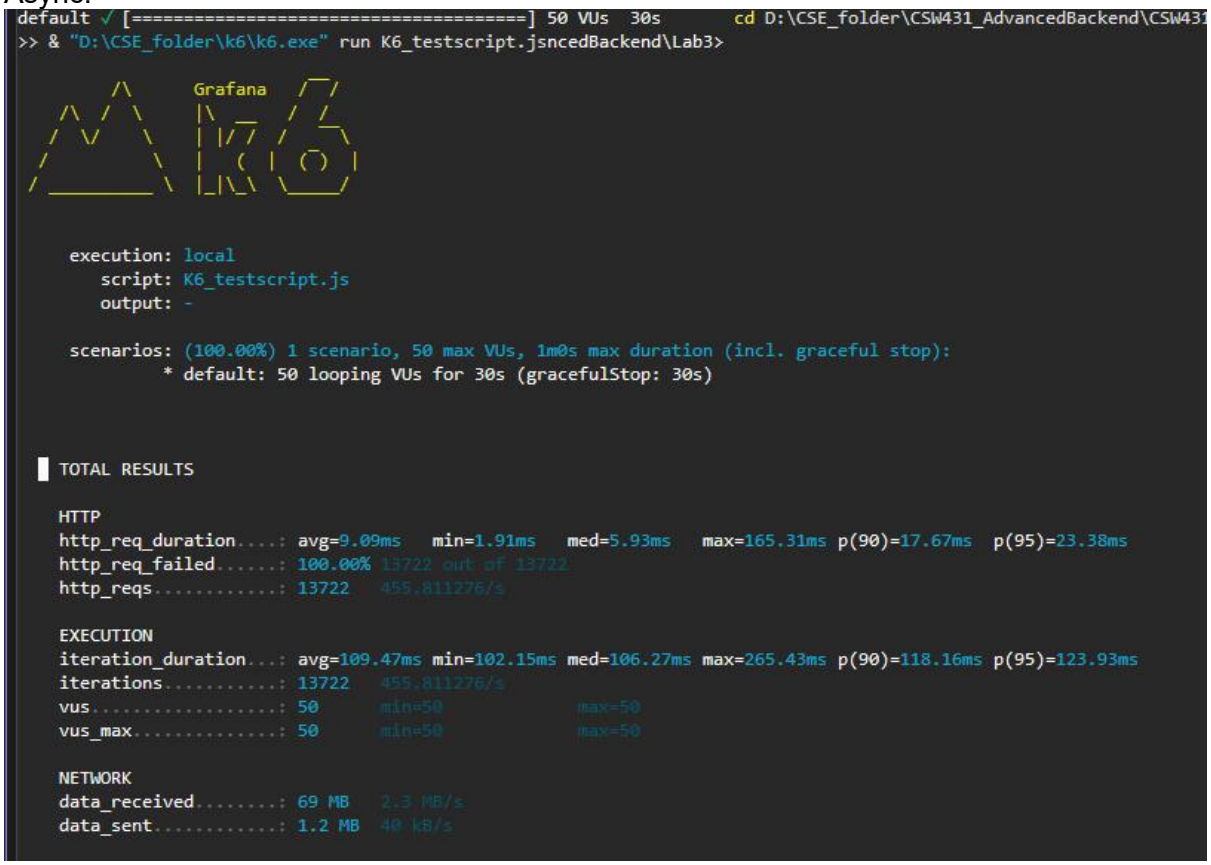
Metric	Avg	Median	p90	p95	Max	Reqs/30s
Sync	8.68 ms	5.26 ms	12.65 ms	17.38 ms	569.92 ms	13781
Async baseline	9.09 ms	5.93 ms	17.67 ms	23.38 ms	165.31 ms	13722

The sync and async baseline endpoints show similar throughput (~459 req/s) because the dataset is small and the DB response time is fast. The key difference is the max latency: sync produced a 569.92 ms spike vs 165.31 ms for async, showing that async better handles the worst-case concurrent requests by not starving the thread pool.

Sync:



Async:



Question 6: Multithreading

- When should multithreading be applied in backend systems?
  In which cases does multithreading reduce performance?

Related Theory

- Multithreading
- Parallelism vs Concurrency

Submission Requirement

- Evaluation of multithreading effectiveness based on measured data

Deliverables

Performance Report (PDF / MD):

- Baseline metrics
- Bottleneck analysis
- Optimization techniques
- Async vs Multithreading comparison

Answer:

When should multithreading be applied in backend systems:

- CPU-bound tasks: computations, image/video processing, cryptographic operations, data transformation where work genuinely runs on CPU cores.

- When tasks are independent and can run in parallel without shared mutable state.

- When the server has multiple CPU cores available and the workload is large enough to justify thread coordination overhead.

When does multithreading reduce performance:

- On I/O-bound tasks: using Task.Run() to run DB queries on separate threads adds thread scheduling overhead without reducing wait time.

- When thread count exceeds CPU cores, excessive context switching consumes CPU time with no productive work.

- When shared resources require locking, contention serializes execution and negates parallelism gains.

Measured results - all four endpoints compared:

Endpoint	Avg	Median	p90	p95	Max	Reqs/30s
Stats Sequential	12.15 ms	8.20 ms	24.4 ms	37.41 ms	104.51 ms	13365
Stats Parallel-Async	52.34 ms	49.41 ms	82.84 ms	94.61 ms	173.12 ms	9844
CPU Sequential	12.49 ms	8.56 ms	25.36 ms	36.49 ms	78.47 ms	13312
CPU Multithreaded	12.5 ms	10.01 ms	22.86 ms	29.56 ms	77.47 ms	13318

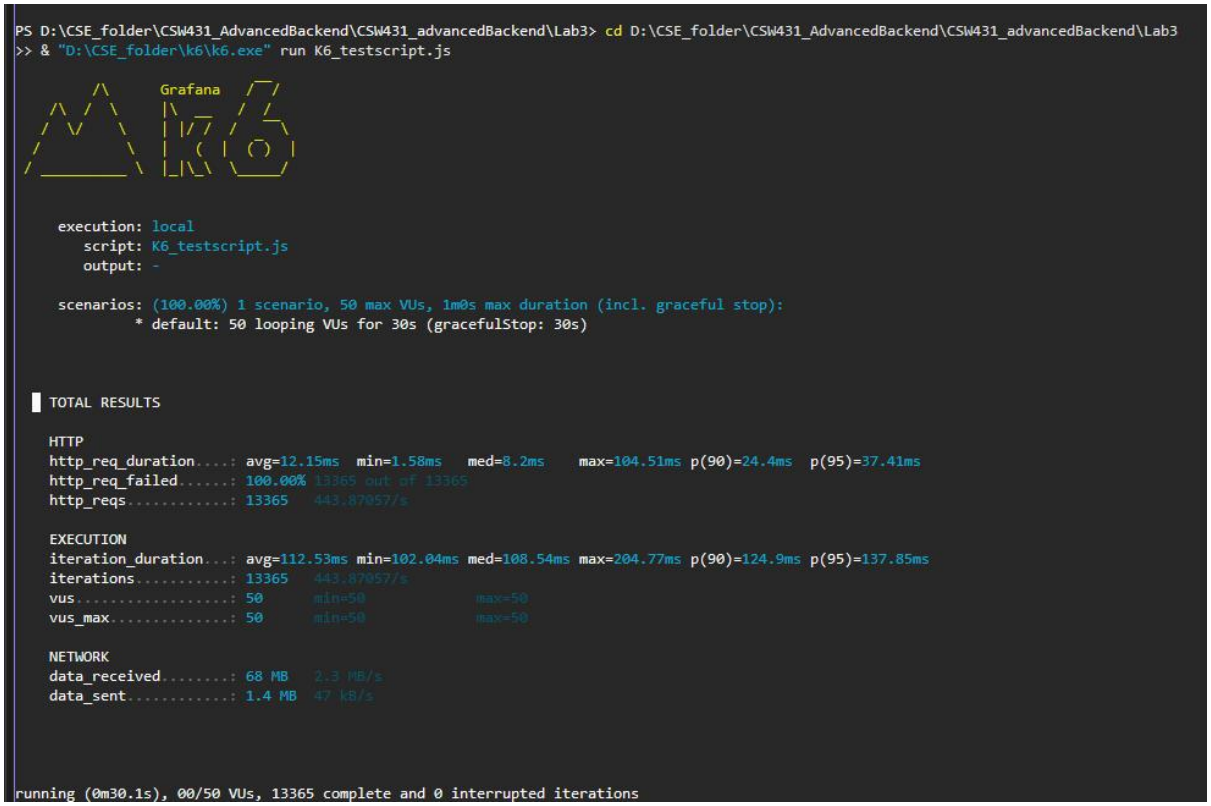
Analysis:

The statistics parallel-async endpoint (/borrowing/parallel-async) performed significantly worse than the sequential version: average latency increased from 12.15 ms to 52.34 ms and total requests dropped from 13,365 to 9,844. This is because the parallel-async endpoint uses Task.WhenAll to run multiple EF Core queries concurrently on the same DbContext instance. EF Core's DbContext is not thread-safe, so concurrent access causes contention and overhead that outweighs any potential gain. For I/O-bound DB queries, sequential async execution is more appropriate.

For the CPU-intensive endpoints, the multithreaded version (Parallel.ForEach) showed nearly identical performance to the sequential version: 12.5 ms vs 12.49 ms average, and 13,318 vs 13,312 requests processed. This indicates that the computation per task is not large enough to benefit from parallelism; the thread coordination overhead cancels out any gains from parallel execution.

- Use async/await for I/O-bound concurrency (DB, network, file).
- Use Parallel.ForEach or Task.Run only for large-scale CPU-bound computations.
- Never use Task.Run to wrap async I/O operations - it adds overhead with no benefit.

Sequential:



Parallel-Async:

```
default ✓ [======================================] 50 VUs  30s       cd D:\CSE_folder\CSW431_AdvancedBackend\CSW431_advancedBackend\Lab3
>> & "D:\CSE_folder\k6\k6.exe" run K6_testscript.jsncedBackend\Lab3>

        /\      Grafana   /‾‾/
     /\  /  \     |\  __   /  /
    /  \/    \    | |/ /  /   ‾‾\
   /          \   |   (  |  (‾)  |
  / __________ \  |_|\_\  \_____/


     execution: local
        script: K6_testscript.js
        output: -

     scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
              * default: 50 looping VUs for 30s (gracefulStop: 30s)


  TOTAL RESULTS

    HTTP
    http_req_duration....: avg=52.34ms  min=2.79ms   med=49.41ms  max=173.12ms p(90)=82.84ms  p(95)=94.61ms
    http_req_failed......: 100.00% 9844 out of 9844
    http_reqs............: 9844    326.697862/s

    EXECUTION
    iteration_duration...: avg=152.74ms min=103.17ms med=149.81ms max=279.45ms p(90)=183.22ms p(95)=194.84ms
    iterations...........: 9844    326.697862/s
    vus..................: 50      min=50          max=50
    vus_max..............: 50      min=50          max=50

    NETWORK
    data_received........: 44 MB   1.5 MB/s
    data_sent............: 1.1 MB  36 kB/s



running (0m30.1s), 00/50 VUs, 9844 complete and 0 interrupted iterations
default ✓ [======================================] 50 VUs  30s
```

Sequentia:

```
default ✓ [======================================] 50 VUs  30s
PS D:\CSE_folder\CSW431_AdvancedBackend\CSW431_advancedBackend\Lab3> cd D:\CSE_folder\CSW431_AdvancedBackend\CSW431_advancedBackend\Lab3
>> & "D:\CSE_folder\k6\k6.exe" run K6_testscript.js

        /\      Grafana   /‾‾/
     /\  /  \     |\  __   /  /
    /  \/    \    | |/ /  /   ‾‾\
   /          \   |   (  |  (‾)  |
  / __________ \  |_|\_\  \_____/


     execution: local
        script: K6_testscript.js
        output: -

     scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
              * default: 50 looping VUs for 30s (gracefulStop: 30s)


  TOTAL RESULTS

    HTTP
    http_req_duration....: avg=12.49ms  min=2.02ms   med=8.56ms   max=78.47ms p(90)=25.36ms p(95)=36.49ms
    http_req_failed......: 100.00% 13312 out of 13312
    http_reqs............: 13312   442.136481/s

    EXECUTION
    iteration_duration...: avg=112.89ms min=102.51ms med=108.94ms max=184.7ms p(90)=125.8ms p(95)=137.04ms
    iterations...........: 13312   442.136481/s
    vus..................: 50      min=50          max=50
    vus_max..............: 50      min=50          max=50

    NETWORK
    data_received........: 67 MB   2.2 MB/s
    data_sent............: 1.5 MB  48 kB/s



running (0m30.1s), 00/50 VUs, 13312 complete and 0 interrupted iterations
default ✓ [======================================] 50 VUs  30s
```

Multithreaded:

```
PS D:\CSE_folder\CSW431_AdvancedBackend\CSW431_advancedBackend\Lab3> cd D:\CSE_folder\CSW431_AdvancedBackend\CSW431_advancedBackend\Lab3
>> & "D:\CSE_folder\k6\k6.exe" run K6_testscript.js

        /\      Grafana   /‾‾/
     /\  /  \     |\  __   /  /
    /  \/    \    | |/ /  /   ‾‾\
   /          \   |   (  |  (‾)  |
  / __________ \  |_|\_\  \_____/


     execution: local
        script: K6_testscript.js
        output: -

     scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
              * default: 50 looping VUs for 30s (gracefulStop: 30s)


  TOTAL RESULTS

    HTTP
    http_req_duration....: avg=12.5ms   min=2.05ms   med=10.01ms  max=77.47ms  p(90)=22.86ms  p(95)=29.56ms
    http_req_failed......: 100.00% 13318 out of 13318
    http_reqs............: 13318   442.40817/s

    EXECUTION
    iteration_duration...: avg=112.88ms min=102.25ms med=110.33ms max=183.44ms p(90)=123.29ms p(95)=129.9ms
    iterations...........: 13318   442.40817/s
    vus..................: 50      min=50            max=50
    vus_max..............: 50      min=50            max=50

    NETWORK
    data_received........: 67 MB   2.2 MB/s
    data_sent............: 1.5 MB  50 kB/s



running (0m30.1s), 00/50 VUs, 13318 complete and 0 interrupted iterations
default ✓ [======================================] 50 VUs  30s
```